

Collision-Risk Triage: Backend Code

TechFest 2026-27 Space Technology Hackathon | Team Jugaad Junction

What this PDF contains

Every backend file used in the project (Python, FastAPI, SQLite), in the order they were built. Each file is shown with the command that creates it in PowerShell. All files live inside Documents\space-triage\backend.

Folder layout

space-triage/	
backend/	
venv/	(created by python -m venv venv)
data.py	fake alert generator, saves alerts.db
risk.py	collision probability, score, tier, explanation
main.py	FastAPI server (alerts, decisions, audit log)
test_risk.py	pytest test
montecarlo.py	Monte Carlo check of the Pc formula
Dockerfile	optional, for one-command demo
alerts.db	created automatically by data.py

Step 0: Setup (one time)

Open PowerShell and run these lines one block at a time:

```
cd $HOME\Documents
mkdir space-triage
cd space-triage
mkdir backend
cd backend
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn numpy scipy pydantic
```

When activation works, (venv) appears at the start of the prompt. If PowerShell says scripts are disabled, run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned and try again.

File 1: data.py (fake data generator)

Create it with notepad data.py, paste, save (Ctrl+S). Then run python data.py. You should see *Done: 50 alerts saved.*

```
import random, sqlite3

def make_alerts(n=50, seed=42):
    random.seed(seed)
    alerts = []
    for i in range(n):
        alerts.append({
            "id": i + 1,
            "object_id": f"{random.randint(2015, 2025)}-{random.randint(1, 99):03d}",
            "miss_distance_m": random.uniform(50, 8000),
            "rel_velocity_kms": random.uniform(0.5, 14),
            "tca_hours": random.uniform(0.5, 72),
            "sigma_m": random.uniform(20, 2000),
            "radius_m": random.uniform(1, 10),
        })
    return alerts

def save_to_db(alerts):
    con = sqlite3.connect("alerts.db")
    con.execute("""CREATE TABLE IF NOT EXISTS alerts(
```

```

        id INTEGER PRIMARY KEY, object_id TEXT, miss_distance_m REAL,
        rel_velocity_kms REAL, tca_hours REAL, sigma_m REAL, radius_m REAL)"""
con.execute("DELETE FROM alerts")
for a in alerts:
    con.execute("INSERT INTO alerts VALUES (?, ?, ?, ?, ?, ?, ?)",
        (a["id"], a["object_id"], a["miss_distance_m"], a["rel_velocity_kms"],
        a["tca_hours"], a["sigma_m"], a["radius_m"]))
con.commit(); con.close()

if __name__ == "__main__":
    save_to_db(make_alerts())
    print("Done: 50 alerts saved")

```

File 2: risk.py (risk engine)

Create it with notepad `risk.py`. It holds the collision probability (Pc) formula, the uncertainty stress test, the combined score, the High/Medium/Low tier and the plain-language explanation.

```

import math

def collision_probability(miss_m, sigma_m, radius_m):
    # Simple 2D Gaussian approximation
    return (radius_m**2 / (2 * sigma_m**2)) * math.exp(-(miss_m**2) / (2 * sigma_m**2))

def stress_test(miss_m, sigma_m, radius_m):
    # How much does Pc change if uncertainty is 0.5x, 1x, 2x?
    pcs = [collision_probability(miss_m, sigma_m * f, radius_m) for f in (0.5, 1, 2)]
    return max(pcs), pcs[1]

def score(a):
    pc = collision_probability(a["miss_distance_m"], a["sigma_m"], a["radius_m"])
    worst_pc, _ = stress_test(a["miss_distance_m"], a["sigma_m"], a["radius_m"])
    pc_part = min(1, math.log10(max(pc, 1e-12)) / -12)
    pc_part = 1 - pc_part
    close_part = max(0, 1 - a["miss_distance_m"] / 5000)
    urgency = max(0, 1 - a["tca_hours"] / 72)
    speed = min(1, a["rel_velocity_kms"] / 14)
    total = 0.45*pc_part + 0.25*close_part + 0.2*urgency + 0.1*speed
    diluted = worst_pc > 10 * pc and pc < 1e-4
    tier = "High" if total > 0.55 else "Medium" if total > 0.35 else "Low"
    return {"pc": pc, "worst_pc": worst_pc, "score": round(total, 3),
            "tier": tier, "diluted": diluted}

def explain(a, r):
    parts = [f"{r['tier']} priority:",
            f"miss distance {a['miss_distance_m']:.0f} m,",
            f"closest approach in {a['tca_hours']:.1f} h,",
            f"collision probability {r['pc']:.2e}."]
    if r["diluted"]:
        parts.append("Warning: uncertainty is large, so the true risk may be higher.")
    return " ".join(parts)

```

Quick test (should print tier High):

```
python -c "from risk import score; print(score({'miss_distance_m':250,'sigma_m':100,'radius_m':5,'tca_hours':2,'rel_velocity_kms':14}))"
```

File 3: main.py (FastAPI server)

Create it with notepad `main.py`. Endpoints: GET `/alerts` (sorted list, optional `?sigma_mult=2` for the what-if slider), POST `/alerts/{id}/decision` (Escalate / Monitor / Dismiss) and GET `/audit` (decision log).

```

import sqlite3
from datetime import datetime
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from risk import score, explain

```

```

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                    allow_methods=["*"], allow_headers=["*"])

def init():
    con = sqlite3.connect("alerts.db")
    con.execute("""CREATE TABLE IF NOT EXISTS audit(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        alert_id INTEGER, action TEXT, time TEXT)""")
    con.commit(); con.close()
init()

def load():
    con = sqlite3.connect("alerts.db")
    con.row_factory = sqlite3.Row
    rows = [dict(r) for r in con.execute("SELECT * FROM alerts")]
    con.close()
    return rows

def latest_decisions():
    con = sqlite3.connect("alerts.db")
    rows = con.execute("SELECT alert_id, action FROM audit ORDER BY id").fetchall()
    con.close()
    return {aid: act for aid, act in rows}

@app.get("/alerts")
def alerts(sigma_mult: float = 1.0):
    dec = latest_decisions()
    out = []
    for a in load():
        a["sigma_m"] *= sigma_mult
        r = score(a)
        out.append(**a, **r, "why": explain(a, r),
                  "decision": dec.get(a["id"], "Pending"))
    return sorted(out, key=lambda x: x["score"], reverse=True)

class Decision(BaseModel):
    action: str

@app.post("/alerts/{alert_id}/decision")
def decide(alert_id: int, d: Decision):
    con = sqlite3.connect("alerts.db")
    con.execute("INSERT INTO audit(alert_id, action, time) VALUES (?, ?, ?)",
                (alert_id, d.action, datetime.now().isoformat()))
    con.commit(); con.close()
    return {"ok": True}

@app.get("/audit")
def audit():
    con = sqlite3.connect("alerts.db")
    rows = con.execute("SELECT alert_id, action, time FROM audit ORDER BY id DESC").fetchall()
    con.close()
    return [{"alert_id": a, "action": b, "time": c} for a, b, c in rows]

```

Start the server (leave this window open):

```
uvicorn main:app --reload
```

Then open <http://127.0.0.1:8000/alerts>, <http://127.0.0.1:8000/docs> and <http://127.0.0.1:8000/audit> in the browser.

File 4: test_risk.py (pytest)

Install with `pip install pytest`, create it with notepad `test_risk.py`, then run `pytest`. A green pass means closer objects score riskier.

```
from risk import collision_probability
```

```
def test_closer_is_riskier():
    assert collision_probability(100, 500, 5) > collision_probability(3000, 500, 5)
```

File 5: montecarlo.py (validation of the Pc formula)

Open a third PowerShell tab, go to the backend folder, activate the venv, then create it with notepad `montecarlo.py` and run `python montecarlo.py`. It simulates 10 million random encounters per case and compares the results with the fast formula and with the exact textbook answer (noncentral chi-square). The formula is accurate when the object is small compared with the position uncertainty; it gets worse when the radius approaches the uncertainty (a known limitation to mention to judges).

```
import numpy as np
from scipy.stats import ncx2
from risk import collision_probability

rng = np.random.default_rng(42)

# (miss distance m, position uncertainty m, object radius m)
cases = [
    (100, 100, 5),
    (250, 100, 5),
    (500, 300, 10),
    (200, 150, 10),
]

BATCH = 1_000_000
BATCHES = 10
N = BATCH * BATCHES

print(f"{'miss':>5} {'sigma':>5} {'R':>3} | {'formula':>9} {'exact':>9} {'simulated':>9} | {'formula vs sim':>14}")
print("-" * 72)

for d, s, r in cases:
    hits = 0
    for _ in range(BATCHES):
        x = rng.normal(d, s, BATCH) # position error along the miss direction
        y = rng.normal(0, s, BATCH) # position error sideways
        hits += np.count_nonzero(x * x + y * y < r * r) # inside the object = collision

    sim = hits / N
    formula = collision_probability(d, s, r)
    exact = ncx2.cdf((r / s) ** 2, 2, (d / s) ** 2)
    err = abs(formula - sim) / sim * 100

    print(f"{d:5d} {s:5d} {r:3d} | {formula:9.2e} {exact:9.2e} {sim:9.2e} | {err:12.1f} %")
```

File 6: Dockerfile (optional)

Needs Docker Desktop. Save as `Dockerfile` (no extension) in the backend folder, then run the two commands below.

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install fastapi uvicorn numpy scipy pydantic
RUN python data.py
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t triage-api .
docker run -p 8000:8000 triage-api
```

Common problems

'pyhton' not recognized	-> typo, the word is: python
No module named numpy/risk	-> venv not active, or not inside the backend folder
Empty alerts	-> run: python data.py (creates alerts.db)
File saved as main.py.txt	-> run: dir, then: ren main.py.txt main.py
Port 8000 busy	-> close old terminals, or stop them with Ctrl+C